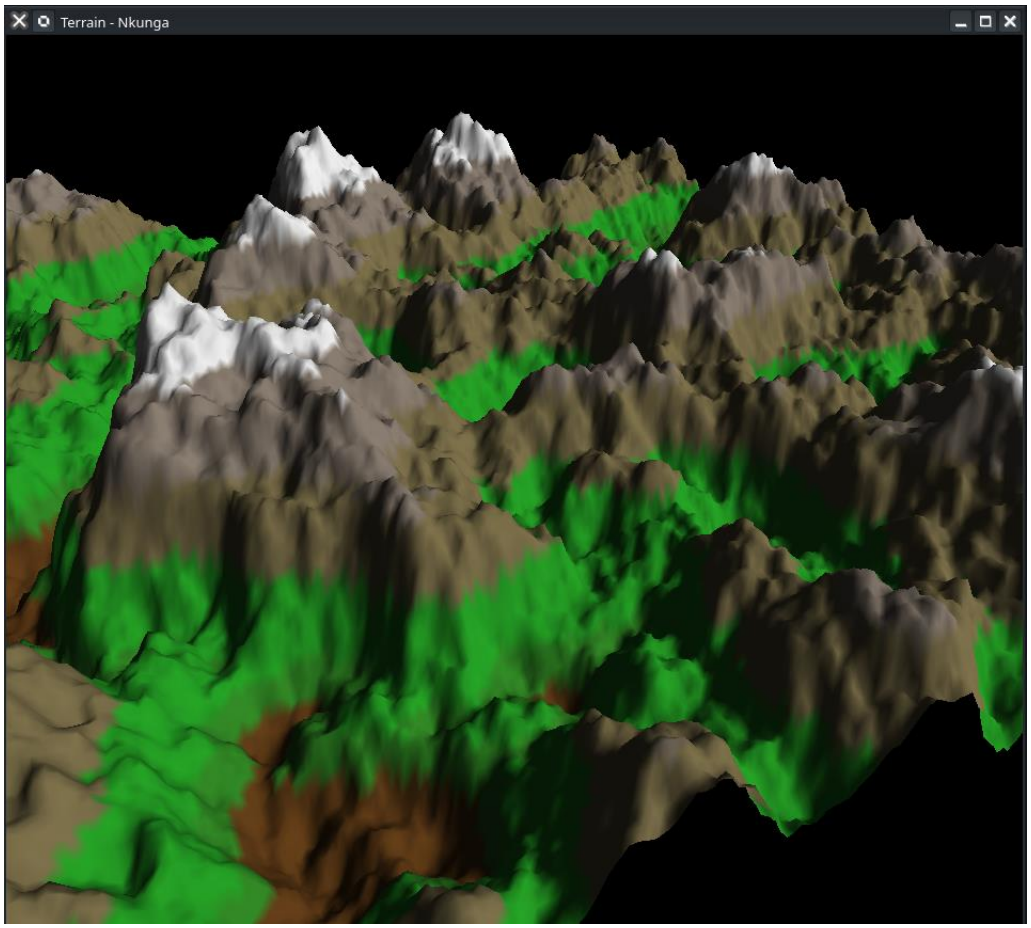
Programming Project Report

Daniel Nkunga
100292653

Problem Statement:

The goal of this programming assignment was to implement basic terrain generation as to be used as the background for a flight simulator. To do this, any terrain generation function could be used and Phong lighting should be used to render lighting on the environment.



Final implementation of terrain generation

Design:

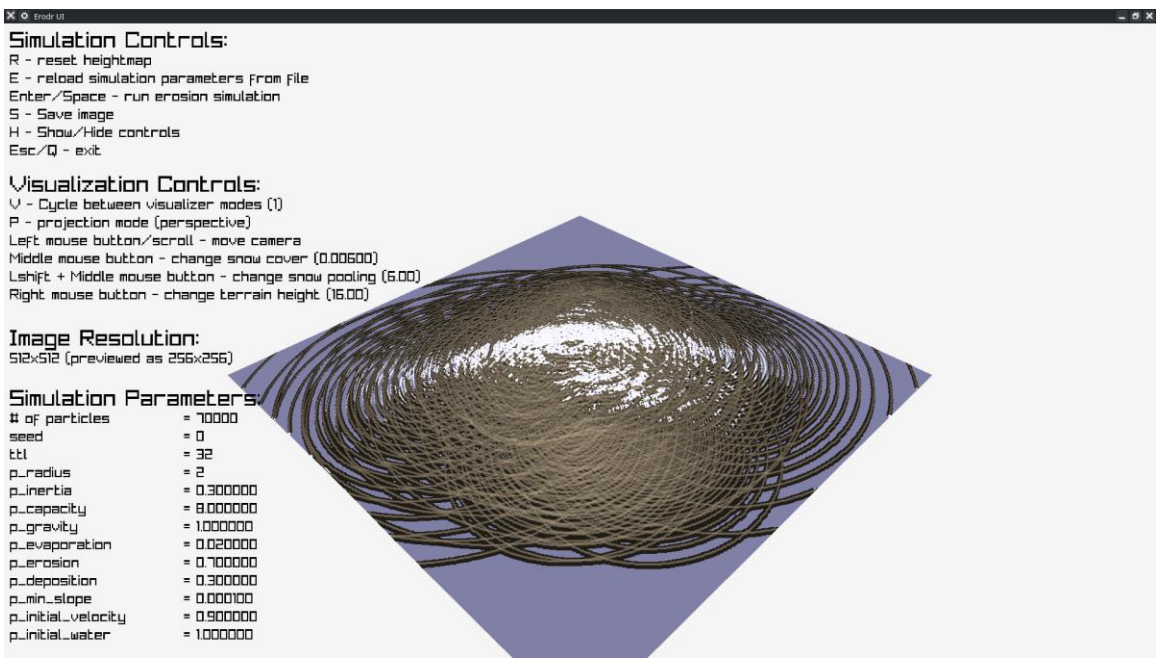
There were to distinct designs of this program: erosion based terrain generation and Perlin noise based terrain generation.

Initially, my goal was to create an erosion-based terrain generation model because I thought it sounded cool. From my casual research before this class, I knew that common terrain generation algorithms were mainly based on generating a height map based on a

random values, or noise. Most commonly, this noise is Perlin noise which, though interesting and accurate looking, is very typical. I had heard of erosion-based terrain generation before and was curious of how to implement it.

I initially was going to read and implement the research paper from Hans Theobald Beyer of *Implementation of a method for hydraulic erosion* but given both a lack of time to properly dive into a research paper and the depth of the topic in general, I moved onto looking for someone else's implementation of erosion terrain generation where I found Henrik A. Glass had the same idea and implemented the paper in a GitHub repo. Glass had implemented his approach in Raylib, a library I had used before, so I was further excited for a hopefully simplistic approach of reading over code and translating one graphics library to another.

I stopped this approach a night of implementation when I realized my LLM usage. I was black boxing a majority of the process, not getting the results Glass showed on his repo in the slightest, and still did not understand the fundamental of the algorithm. Furthermore, the last prompt I had just made me generate Perlin noise. Though this approach is interesting, I set it aside for the moment so, when I come back to it, I could understand it thoroughly and not just implement it half haphazardly.



Henrik A. Glass terrain example; there may be an error with the way I'm running it

This meant going back to a noise-based terrain generation approach where I committed to going back to the fundamentals and using Perlin noise. To learn what was going on, I rewatched a video I had previously watched for fun by Acerola: Scuplting Terrain With Math. Though it relied heavily on Perlin noise, it was the night before the assignment, and I didn't have the most time to watch their video on it.

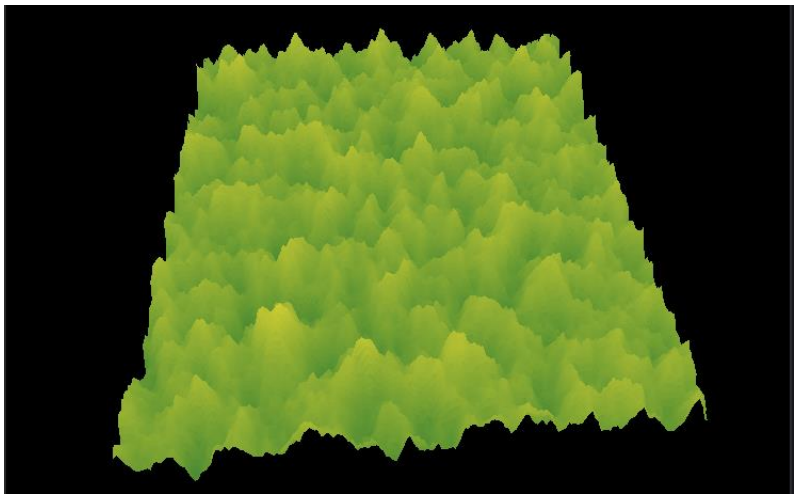
In the end, this algorithm could boil down to just generating heigh off random noise, but for more detailed approaches, a summation of heights is added with decreasing amplitudes and increasing frequencies over a set interval. The result is terrain that has both large landmasses and smaller details on said landmasses.

Structurally, the code follows many of the same principles from previous coding assignments. The terrain is a massive plane of GL triangles generated using GL_TRIANGLE_STRIP. Outside of basic OpenGL commands, many of the terrain generation functions are also housed in the main file for simplicity.

Implementation:

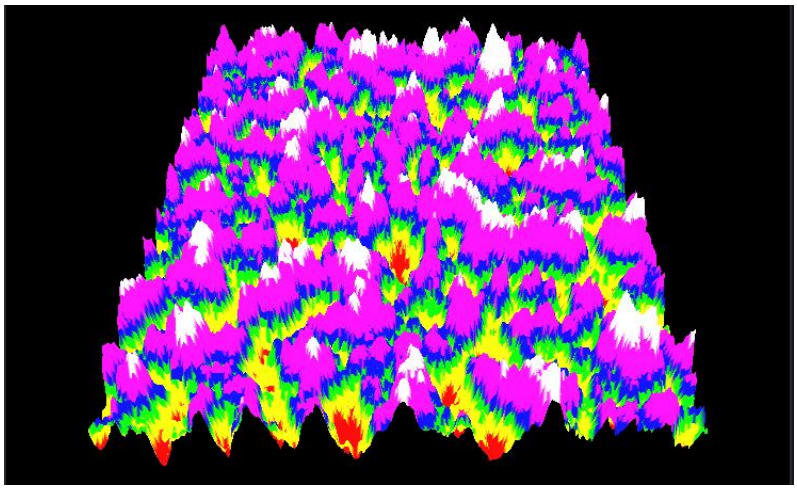
Again due to poor time management on my end, a lot of LLM usage was done for this assignment.

The first part of the project came through the implementation of Perlin noise through a simple enough generation function. Then a simple plane of triangles was created. Finally, the noise was used to modify the height of the triangles to make terrain.



Initial generation of terrain

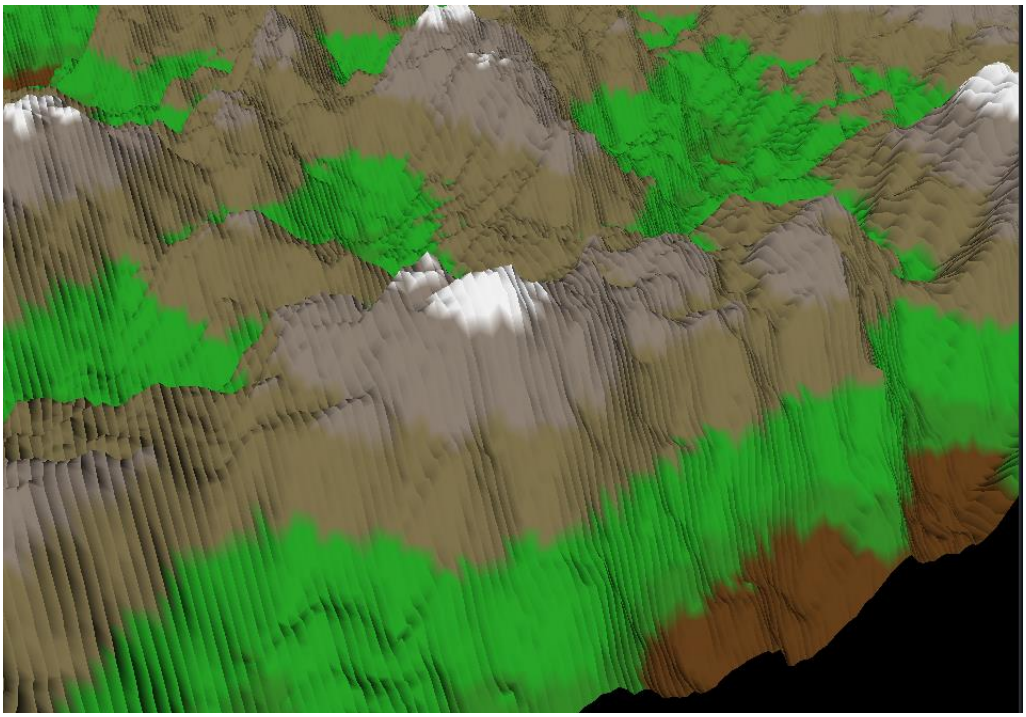
Afterwards, more dynamic color mapping was done to fufill that requirement of the assignment and parameters were tweaked for the terrain. This is also notably where I realized I would be submitting late and took the time to make the program more interesting while I was under less pressure. This main came down to adding basic keyboard controls to regenerate the terrain or change the scale and octaves for the Perlin noise.



Terrain after messing with more parameters and going crazy with colors

Finally, I began implementing Phong light and the final touches for controls. At its simplest, Phong lighting will take one light source and modify the color of terrain. In this case, my light source was white so modifying the color just meant linearly changing the RGB value depending on its location relative to the light source.

Testing:
The program was tested incrementally during development. Notable bugs and problems came though messing with the Perlin noise generation. Initially, there were also controls for persistence and lacunarity, but they had little effects so I decided to remove them. This does make sense because they would add more smaller details that would be harder to see without either increasing the size of the scene or zooming in the camera. Another possible bug comes with the implementation and turning off of phone lighting. During initial testing, my camera was very zoomed out. When I implemented the method to turn of Phong lighting, I noticed weird lighting artifacts most likely due to the octaves when generating Perlin noise. I am unsure if these were always present and just unnoticeable from a distance or if I somehow am causing them when implementing the lighting.



Graphical artifacts when rendering without lighting; most likely from Perlin noise octaves being too high for such a small resolution [untested theory]

Conclusions:

The resulting terrain was satisfactory, and the project was very informative about how annoying complex simple task can be. If I were to redo the assignment again, I would like to take more time to fully implement Perlin noise on my own. Its a relatively simple algorithm to reimplement with a lot of tutorials and documentation and, once generated, it can be used quite often to get random looking data. I would like to also commit to starting earlier, but given I'm one for three in working on these programming assignments during a reasonable amount of time, I would tell myself to just focus on the simple implementation and make the project "interesting" though embellishments, not to try and jump of the deep end while barely knowing how to swim.